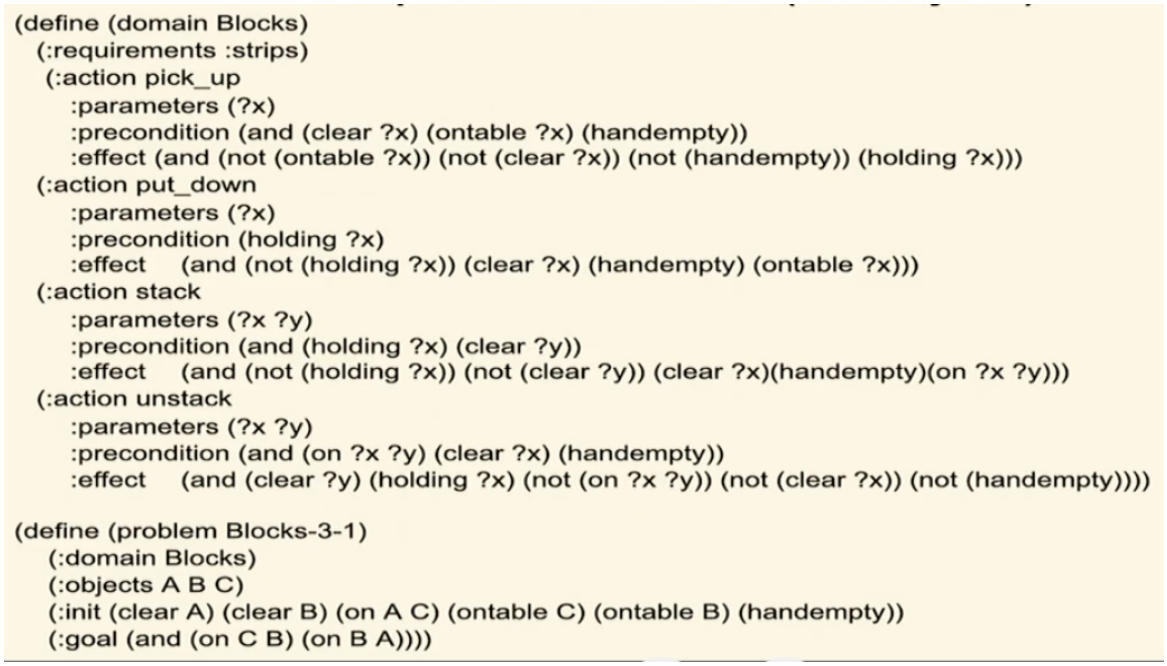
- Come algoritmo di ricerca, però non abbiamo degli stati, però siamo in dei numeri praticamente
- Noi dobbiamo qui definire il dominio
  - Quali azioni e sensori e come funzionano
  - Qual'è l'obiettivo e la soluzione iniziale
- Qui l'agente ottiene informazioni attraverso le percezione, apprendimento e linguaggio
  - L'agente deve sapere le informazioni
  - E siccome le informazioni sono tanti e complicate, si tiene tutto in una
    - **Base di conoscenza**
    - **Algoritmo di inferenza**
      - Questo lavora sulla base di conoscenza per ottenere informazioni
      - Meccanismo di inferenza
      - Qui esempio c'è la logica proposizionale, logica di primo ordine
- Rappresentiamo pacman con delle variabili/proposizioni
  - Definizione variabili
    - Localizzatori di muri
    - Muro*0,1 => c'è un muro a 0, 1 esempio*
    - Percettori
    - Blocked_W_0 => E' bloccato a destra, non può andare a destra, e questoa. Tempo 0
    - Azioni
    - W_0 => pacman si muove a destra a tempo 0
    - Pacman locazione
    - At_0,0_0 => pacman è a 0,0 al tempo 0
    Notiamo: precede una variabile
  - E da questa notazione di come sono fatte le variabili, possiamo definire una base di conoscenza
    - Wall_0,0 ^ wall_0,1 ^ ...
  - Ora però dobbiamo creare una funzione di successione di stato
    - "se sono a stato X, e faccio azione W, per via degli stati Muro, allora Blocked quindi
    - Strips, dato un problema $P = (f, O, I, G)$
      - F = stati (atomi) (solo veri e farsi)
      - O = Operazioni
      - Date delle pre condizioni, può aggiungere o rimuovere dalla lista
      - Pre condizioni = certi stati sono veri oppure falsi
      - E da queste operazioni, lavora sulla lista
      - I = Stato iniziale (fa parte di F)

- G = Goal (fa parte di F)

Problema: ci sono tante ripetizioni nelle variabili, e per questo che si è creato il PDDL

- PDDL



```
(define (domain Blocks)
  (:requirements :strips)
   (:action pick_up
      :parameters (?x)
      :precondition (and (clear ?x) (ontable ?x) (handempty))
      :effect (and (not (ontable ?x)) (not (clear ?x)) (not (handempty)) (holding ?x)))
  (:action put_down
      :parameters (?x)
      :precondition (holding ?x)
      :effect    (and (not (holding ?x)) (clear ?x) (handempty) (ontable ?x)))
  (:action stack
      :parameters (?x ?y)
      :precondition (and (holding ?x) (clear ?y))
      :effect    (and (not (holding ?x)) (not (clear ?y)) (clear ?x)(handempty)(on ?x ?y)))
  (:action unstack
      :parameters (?x ?y)
      :precondition (and (on ?x ?y) (clear ?x) (handempty))
      :effect    (and (clear ?y) (holding ?x) (not (on ?x ?y)) (not (clear ?x)) (not (handempty))))

(define (problem Blocks-3-1)
    (:domain Blocks)
    (:objects A B C)
    (:init (clear A) (clear B) (on A C) (ontable C) (ontable B) (handempty))
    (:goal (and (on C B) (on B A))))
```

- Define (domain => qui definiamo il dominio
- Definiamo le variabili con ? Prima del nome
- Nota: lavoriamo sempre su atomi

Ora però nasce un problema: come possiamo sapere qual'è una soluzione desiderata?

- Ricerca euristica
  - Facciamo versione semplificate per fare l'euristica
  - E poi risolviamo/ci avviciniamo attraverso le versioni più semplificate
- Pianificazione come SAT
  - rendiamo il piano in un problema di soddisfacimento

    Notiamo qualcosa: qui per creare la funzione di successione, siccome abbiamo tante variabili, diventa impossibile, e qui nascono gli algoritmi di planning

Esempio:

Problem $P = (F, I, O, G)$ where:

- $F = \{p, q, r\}$
- $I = \{p\}$
- $G = \{q, r\}$
- O has two actions a and b such that:

  $Action\ 1: Prec\ (a) = \{p\}, Add(a) = \{q\}, Del(a) = \{\}$
  $Action\ 2: Prec\ (b) = \{q\}, Add(b) = \{r\}, Del(b) = \{q\}$

- G = {q, r} => sia q che r siano veri
- Action 1: la precondizione è che p è vera, se p è vera allora aggiungiamo q e non rimuoviamo niente

  Un altro esempio più complicato:

- $F = \{p1ATrA, p1ATrB, p2ATrA, p2ATrB, robotATrA, robotATrB, p1held, p2held\}$
- $I = \{p1ATrA, p2ATrA, robotATrB\}$
- $G = \{p1ATrB, p2ATrB\}$
- O contains 10 actions:
  - Pick-p1rA: Prec = {p1ATrA, robotATA}, Add = {p1held}, Del = {p1ATrA}
  - Pick-p1rB: Prec = {p1ATrB, robotATB}, Add = {p1held}, Del = {p1ATrB}
  - Drop-p1rA: Prec = {p1held, robotATA}, Add = {p1ATrA}, Del = {p1held}
  - Drop-p1rB: Prec = {p1held, robotATB}, Add = {p1ATrB}, Del = {p1held}
  - Move-A-to-B: Prec = {robotATA}, Add = {robotATB}, Del = {robotATA}
  - Move-B-to-A: Prec = {robotATB}, Add = {robotATA}, Del = {robotATB}
  - Pick-p2rA: Prec = {p2ATrA, robotATA}, Add = {p2held}, Del = {p2ATrA}
  - Pick-p2rB: Prec = {p2ATrB, robotATB}, Add = {p2held}, Del = {p2ATrB}
  - Drop-p2rA: Prec = {p2held, robotATA}, Add = {p2ATrA}, Del = {p2held}
  - Drop-p2rB: Prec = {p2held, robotATB}, Add = {p2ATrB}, Del = {p2held}

- robotATrA => robot è nella stanza A
- p1ATra => pacco 1 è nella stanza A
- P1helf => il robot tiene il paccho 1
- Il nostro obiettivo è portare p1ATrB p2ATrB, cioè entrambi i pacchi stanza B